

Testing Guide - MorphoScan Pro

Overview

This guide provides comprehensive information about the testing infrastructure, strategies, and best practices for MorphoScan Pro. The project uses Vitest as the testing framework with extensive unit and integration test coverage.

Table of Contents

1. [Test Infrastructure](#)
 2. [Running Tests](#)
 3. [Test Coverage](#)
 4. [Unit Tests](#)
 5. [Integration Tests](#)
 6. [Writing Tests](#)
 7. [Mocking Strategies](#)
 8. [Best Practices](#)
 9. [Troubleshooting](#)
-

Test Infrastructure

Framework & Tools

- **Test Runner:** Vitest v4.0.15
- **Testing Library:** @testing-library/react
- **Test Environment:** jsdom
- **Coverage Tool:** v8/istanbul (built into Vitest)

Configuration

The main configuration file is `vitest.config.ts` :

```
{
  test: {
    environment: "jsdom",
    setupFiles: ["/src/test/setup.ts"],
    globals: true,
    include: ["src/**/*.{test,spec}.{ts,tsx}"],
    coverage: {
      reporter: ["text", "json", "html"],
      thresholds: {
        branches: 80,
        functions: 80,
        lines: 80,
        statements: 80
      }
    }
  }
}
```

Test Setup File

Located at `src/test/setup.ts`, this file:

- Extends Vitest's expect with jest-dom matchers
- Mocks browser APIs (matchMedia, IntersectionObserver, ResizeObserver)
- Sets up cleanup after each test
- Provides test utilities and helpers

Running Tests

Available Commands

```
# Run all tests in watch mode (interactive)
npm run test

# Run all tests once (CI mode)
npm run test:run

# Run tests with UI interface
npm run test:ui

# Generate coverage report
npm run test:coverage

# Run end-to-end tests (Playwright)
npm run test:e2e

# Run all tests (unit + e2e)
npm run test:all
```

Running Specific Tests

```
# Run tests in a specific file
npx vitest run src/lib/__tests__/security.test.ts

# Run tests matching a pattern
npx vitest run --grep "RateLimiter"

# Run tests in watch mode for specific file
npx vitest watch src/hooks/__tests__/useOfflineSync.test.ts
```

Test Coverage

Coverage Goals

The project maintains the following coverage thresholds:

- **Branches:** 80%
- **Functions:** 80%
- **Lines:** 80%
- **Statements:** 80%

Viewing Coverage Reports

After running `npm run test:coverage`, view the HTML report:

```
# Open coverage report in browser
open coverage/index.html
```

Coverage Exclusions

The following files/directories are excluded from coverage:

- `node_modules/`
- `src/test/` - Test utilities
- `**/*.d.ts` - Type definitions
- `**/index.ts` - Re-export files
- `**/*.config.*` - Configuration files
- `src/main.tsx` - Application entry point
- `src/vite-env.d.ts` - Vite types

Unit Tests

Critical Modules Covered

1. Security Module (`src/lib/__tests__/security.test.ts`)

Tests for `src/lib/security.ts`:

Components Tested:

- `InputSanitizer` - Input validation and sanitization
- `RateLimiter` - Rate limiting functionality

- DataEncryptor - AES-GCM encryption/decryption
- SecureStorage - Encrypted localStorage wrapper
- generateCSPHeader() - CSP header generation
- initializeSecurity() - Security initialization

Coverage:

- 30+ test cases
- Edge cases for XSS prevention
- Encryption key management
- Rate limit enforcement

Example Test:

```
it("should sanitize HTML tags from input", () => {
  const input = "<script>alert('xss')</script>Hello";
  const result = InputSanitizer.sanitizeString(input);
  expect(result).not.toContain("<");
  expect(result).not.toContain(">");
});
```

2. Offline Sync Hook (src/hooks/__tests__/useOfflineSync.test.ts)

Tests for src/hooks/useOfflineSync.ts :

Functionality Tested:

- Queue operations (insert, update, delete)
- Sync with Supabase when online
- Exponential backoff retry logic
- localStorage persistence
- Online/offline event handling
- Sync statistics tracking

Coverage:

- 20+ test cases
- Network failure scenarios
- Concurrent operations
- Rate limiting

3. NSFW Cache (src/addons/nsfw-scanner/scanner/__tests__/nsfwCache.test.ts)

Tests for src/addons/nsfw-scanner/scanner/nsfwCache.ts :

Functionality Tested:

- LRU cache eviction
- SHA-256 image hashing
- Cache expiration (1 hour TTL)
- Max cache size enforcement (100 entries)
- Cache statistics

Performance Gains:

- ~50-200ms saved per cached result
- Reduces ML inference load

4. DLC Rate Limiter (src/dlc/security/__tests__/RateLimiter.test.ts)

Tests for src/dlc/security/RateLimiter.ts :

Singleton Instances:

- `licenseValidationLimiter` - 5 requests/minute
- `nsfwDetectionLimiter` - 20 requests/minute
- `ageVerificationLimiter` - 3 requests/5 minutes

Coverage:

- 30+ test cases
- Time window expiration
- Reset functionality
- Concurrent access

5. Content Package Management (`src/lib/__tests__/contentPackage.test.ts`)

Tests for `src/lib/contentPackage.ts` :

Functionality Tested:

- Package download with progress tracking
- SHA-256 checksum verification
- Package extraction
- Version management
- Installation tracking
- `localStorage` persistence

Coverage:

- 25+ test cases
- Network failure handling
- Large file handling
- Concurrent downloads

Integration Tests

User Flow Tests

Located in `src/__tests__/integration/` , these tests cover complete user workflows:

1. DLC Purchase Flow (`dLcPurchaseFlow.test.ts`)**Flow Tested:**

1. Browse DLC catalog
2. Select package
3. Process payment
4. Verify license
5. Download content
6. Install package

Scenarios:

- Successful purchase
- Payment failures
- Network interruptions
- Duplicate purchase prevention

2. NSFW Detection with Web Worker (`nsfwDetectionWebWorker.test.ts`)

Flow Tested:

1. Initialize web worker
2. Load NSFW model
3. Detect content (neutral/suggestive/explicit)
4. Cache results
5. Handle errors

Performance:

- 75% faster with web worker
- Non-blocking UI
- Rate limiting enforcement

3. Age Verification (`ageVerification.test.ts`)

Flow Tested:

1. Prompt for age verification
2. Validate birthdate
3. Grant/deny access
4. Persist verification
5. Clear on logout

Scenarios:

- Valid verification (18+)
- Underage rejection (<18)
- Invalid inputs
- Rate limiting (3 attempts/5 min)

4. License Activation (`licenseActivation.test.ts`)

Flow Tested:

1. Enter license key
2. Validate with server
3. Register device
4. Unlock features
5. Sync across devices

Scenarios:

- Valid activation
- Invalid keys
- Expired licenses
- Device limit enforcement
- Offline validation

Writing Tests

Test Structure

Follow the AAA (Arrange-Act-Assert) pattern:

```
describe("MyComponent", () => {
  beforeEach(() => {
    // Arrange: Setup
    localStorage.clear();
    vi.clearAllMocks();
  });

  it("should do something", () => {
    // Arrange: Prepare data
    const input = "test";

    // Act: Execute function
    const result = myFunction(input);

    // Assert: Verify result
    expect(result).toBe("expected");
  });
});
```

Testing React Hooks

Use `@testing-library/react`'s `renderHook`:

```
import { renderHook, waitFor, act } from "@testing-library/react";

it("should update state", async () => {
  const { result } = renderHook(() => useMyHook());

  act(() => {
    result.current.updateValue("new value");
  });

  await waitFor(() => {
    expect(result.current.value).toBe("new value");
  });
});
```

Testing Async Functions

```
it("should handle async operations", async () => {
  const promise = myAsyncFunction();

  await expect(promise).resolves.toBe("success");
  // or
  const result = await myAsyncFunction();
  expect(result).toBe("success");
});
```

Testing Errors

```
it("should throw error on invalid input", () => {
  expect(() => {
    dangerousFunction(null);
  }).toThrow("Invalid input");
});

// Async errors
it("should reject with error", async () => {
  await expect(asyncFunction()).rejects.toThrow("Error message");
});
```

Mocking Strategies

Mocking Modules

```
// Mock entire module
vi.mock("@/integrations/supabase/client", () => ({
  supabase: {
    from: vi.fn(),
    auth: {
      getUser: vi.fn(),
    },
  },
}));

// Mock specific exports
vi.mock("sonner", () => ({
  toast: {
    success: vi.fn(),
    error: vi.fn(),
  },
}));
```

Mocking Functions

```
// Create mock function
const mockFn = vi.fn();

// Mock implementation
mockFn.mockImplementation(() => "mocked result");

// Mock return value
mockFn.mockReturnValue("result");

// Mock resolved value (Promise)
mockFn.mockResolvedValue("async result");

// Mock rejected value
mockFn.mockRejectedValue(new Error("error"));
```


Mocking Browser APIs

```
// Mock fetch
global.fetch = vi.fn().mockResolvedValue({
  ok: true,
  json: async () => ({ data: "test" }),
});

// Mock localStorage
Storage.prototype.setItem = vi.fn();
Storage.prototype.getItem = vi.fn();

// Mock Date.now
vi.spyOn(Date, "now").mockReturnValue(1234567890);
```

Mocking Timers

```
beforeEach(() => {
  vi.useFakeTimers();
});

afterEach(() => {
  vi.useRealTimers();
});

it("should execute after delay", () => {
  const callback = vi.fn();
  setTimeout(callback, 1000);

  vi.advanceTimersByTime(1000);
  expect(callback).toHaveBeenCalled();
});
```

Best Practices

1. Test Isolation

- Each test should be independent
- Use `beforeEach` to reset state
- Clear mocks between tests
- Avoid shared mutable state

2. Descriptive Test Names

```
// ❌ Bad
it("works", () => { ... });

// ✅ Good
it("should sanitize HTML tags from user input", () => { ... });
```

3. Test One Thing

```
// ❌ Bad - testing multiple things
it("should do everything", () => {
  expect(fn1()).toBe(true);
  expect(fn2()).toBe(false);
  expect(fn3()).toBe("value");
});

// ✅ Good - focused tests
it("should return true for valid input", () => {
  expect(fn1()).toBe(true);
});

it("should return false for invalid input", () => {
  expect(fn2()).toBe(false);
});
```

4. Use Meaningful Assertions

```
// ❌ Bad
expect(result).toBeTruthy();

// ✅ Good
expect(result).toBe(true);
expect(array).toHaveLength(3);
expect(object).toEqual({ key: "value" });
```

5. Test Edge Cases

- Empty inputs
- Null/undefined values
- Maximum/minimum values
- Error conditions
- Boundary conditions

6. Avoid Implementation Details

Test behavior, not implementation:

```
// ❌ Bad - testing implementation
expect(component.state.counter).toBe(1);

// ✅ Good - testing behavior
expect(screen.getByText("Count: 1")).toBeInTheDocument();
```

Troubleshooting

Common Issues

1. Tests Timeout

Problem: Tests hang or timeout

Solutions:

```
// Increase timeout for specific test
it("slow test", async () => {
  // test code
}, 10000); // 10 seconds

// Or use vi.mock to avoid real network calls
vi.mock("@integrations/supabase/client");
```

2. Act Warnings**Problem:** "Warning: not wrapped in act(...)"**Solution:**

```
import { act } from "@testing-library/react";

act(() => {
  result.current.updateValue("new");
});
```

3. Module Not Found**Problem:** Cannot find module '@/...'**Solution:** Check `vitest.config.ts` has correct alias:

```
resolve: {
  alias: {
    "@": path.resolve(__dirname, "./src"),
  },
}
```

4. localStorage/IndexedDB Errors**Problem:** localStorage is not defined**Solution:** Use jsdom environment (already configured):

```
// In vitest.config.ts
test: {
  environment: "jsdom"
}
```

5. Crypto API Errors**Problem:** crypto.subtle is undefined**Solution:** The jsdom environment should provide this, but you can polyfill:

```
// In test setup
global.crypto = {
  subtle: webcrypto.subtle,
  getRandomValues: (arr) => webcrypto.getRandomValues(arr),
};
```

CI/CD Integration

GitHub Actions Example

```
name: Tests

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '22'

      - name: Install dependencies
        run: npm ci

      - name: Run tests
        run: npm run test:run

      - name: Generate coverage
        run: npm run test:coverage

      - name: Upload coverage
        uses: codecov/codecov-action@v3
        with:
          files: ./coverage/coverage-final.json
```

Test Statistics

Current Coverage (Phase 5)

- **Test Files:** 25 total
- **Test Cases:** 269 total
- **Pass Rate:** 97.4% (262 passed, 7 failed)
- **Unit Tests:** 240+ test cases
- **Integration Tests:** 29+ test cases

Critical Modules

Module	Tests	Coverage
security.ts	30+	High
useOfflineSync.ts	20+	High
nsfwCache.ts	16+	High
RateLimiter.ts	31+	High
contentPackage.ts	25+	High

Future Improvements

- 1. Increase E2E Coverage**
 - Add more Playwright tests
 - Test critical user journeys end-to-end
- 2. Visual Regression Testing**
 - Implement screenshot comparison
 - Catch UI regressions automatically
- 3. Performance Testing**
 - Add performance benchmarks
 - Monitor test execution time
- 4. Mutation Testing**
 - Use mutation testing to verify test quality
 - Ensure tests catch real bugs
- 5. Test Data Factories**
 - Create test data builders
 - Simplify test setup

Resources

- [Vitest Documentation](https://vitest.dev/) (https://vitest.dev/)
 - [Testing Library](https://testing-library.com/) (https://testing-library.com/)
 - [Playwright](https://playwright.dev/) (https://playwright.dev/)
 - [Testing Best Practices](https://kentcdodds.com/blog/common-mistakes-with-react-testing-library) (https://kentcdodds.com/blog/common-mistakes-with-react-testing-library)
-

Support

For questions or issues with tests:

1. Check this guide
 2. Review existing test examples
 3. Consult Vitest documentation
 4. Ask in team discussions
-

Last Updated: Phase 5 - December 2025

Maintained By: MorphoScan Pro Development Team